Atelier de programmation

Exercices sur les chapitres 1 à 4

Des programmes courts, des exemples à suivre à la main,
puis quelques problèmes à résoudre ensemble.

Python, collections, complexité et objets

Un exemple avant le programme

Pour chaque problème, nous suivrons la même démarche.

1. Trouver le résultat sur un petit exemple.
2. Expliquer les étapes en français : c'est le pseudocode.
3. Écrire le programme, puis suivre ses variables à la main.
4. Tester un cas normal et un cas particulier.

Les corrigés se trouvent après les questions. Prenez le temps d'essayer avant de passer à la suite.

Nous utiliserons les outils du cours : boucles, fonctions, listes, dictionnaires, ensembles et classes. Aucun module à importer.

Lecture : combien de « e » ?

Sans exécuter le code, prévoyez l'affichage. Notez la valeur de `total` après chaque lettre.

```
def compter_e(texte):
    total = 0
    for lettre in texte:
        if lettre == "e":
            total = total + 1
    return total

print(compter_e("cerise"))
```

Que renvoie la fonction pour `"chat"` ? Et pour `""` ? Le `E` majuscule compte-t-il ?

Corrigé : le compteur augmente deux fois

Lettre	c	e	r	i	s	e
total	0	1	1	1	1	2

L'affichage est 2. La condition est vraie seulement pour les deux lettres `"e"`.

`compter_e("chat")` et `compter_e("")` donnent `0`. Le test distingue `"E"` de `"e"`.

Pour une chaîne de n caractères, la boucle fait n tours : temps $O(n)$. Elle conserve un compteur : mémoire supplémentaire $O(1)$.

À modifier : faites compter une lettre passée en deuxième paramètre, avec `compter(texte, cible)`. Seule la comparaison change.

Débogage : la recherche s'arrête trop tôt

`contient(valeurs, cible)` doit répondre `True` si la cible est dans la liste, sinon `False`.

```
def contient(valeurs, cible):
    for valeur in valeurs:
        if valeur == cible:
            return True
        return False
```

Essayez à la main `contient([3, 8, 5], 8)`.

1. Quelle valeur la boucle lit-elle en premier ?
2. Quelle ligne arrête la fonction ?
3. Où placer `return False` pour examiner toute la liste ?

Corrigé : conclure après la recherche

```
def contient(valeurs, cible):
    for valeur in valeurs:
        if valeur == cible:
            return True
    return False
```

Pour chercher 8, on lit 3, puis 8. La fonction renvoie alors `True`. Pour chercher 9, on lit les trois nombres avant de renvoyer `False`.

Testez aussi `[]`, une cible en première position et une cible en dernière position.

Au mieux, une comparaison suffit : $O(1)$. Au pire, on examine n valeurs : $O(n)$. La fonction utilise $O(1)$ de mémoire supplémentaire.

Complexité : compter les passages

Pour chaque extrait, combien d'appels à `print` pour $n = 3$? Puis pour n quelconque ?

```
# A
print(n)

# B
for i in range(n):
    print(i)

# C
for i in range(n):
    for j in range(n):
        print(i, j)
```

On compte ici chaque affichage comme une opération. Associez $O(1)$, $O(n)$ et $O(n^2)$ aux trois extraits.

Corrigé : 1, n ou n fois n

Extrait	Pour n = 3	Pour n quelconque	Coût
A	1	1	$O(1)$
B	3	n	$O(n)$
C	9	$n \times n$	$O(n^2)$

Dans C, pour chaque valeur de `i`, la boucle sur `j` fait n tours.

Si on double n, B fait deux fois plus de passages. C en fait quatre fois plus.

Petite variante : si la boucle intérieure utilise `range(3)`, on compte $3n$ passages. Le coût est alors $O(n)$.

Lecture de code : une lettre après l'autre

Sans exécuter le programme, donnez la liste et le dictionnaire renvoyés. Tracez `ordre` et `comptes` après chaque caractère de `"aba cab"`.

```
def resume_lettres(texte):
    ordre, comptes = [], {}
    for lettre in texte:
        if lettre != " ":
            if lettre not in comptes:
                ordre.append(lettre)
            comptes[lettre] = comptes.get(lettre, 0) + 1
    return ordre, comptes

print(resume_lettres("aba cab"))
```

Que renvoie la fonction pour une chaîne vide ? Modifie-t-elle la chaîne reçue ?

Corrigé : ordre de découverte et fréquences

Caractère lu	Liste ordre	Dictionnaire comptes
a	`["a"]`	`a: 1`
b	`["a", "b"]`	`a: 1, b: 1`
a	`["a", "b"]`	`a: 2, b: 1`
espace	inchangée	inchangé
c	`["a", "b", "c"]`	`a: 2, b: 1, c: 1`
a puis b	inchangée	`a: 3, b: 2, c: 1`

La fonction renvoie un tuple contenant la liste et le dictionnaire. Pour `""`, elle renvoie `([], {})`. Elle ne modifie pas l'entrée.

Pour n caractères et k caractères distincts conservés : temps $O(n)$ en moyenne et sortie de taille $O(k)$.

Débogage : une recherche trop pressée

Contrat : renvoyer l'indice de la première occurrence de `cible`, ou `None` si elle est absente. La fonction doit accepter une liste vide.

```
def chercher(valeurs, cible):
    for i in range(len(valeurs) - 1):
        if valeurs[i] == cible:
            return i
        return None
```

Essai	Résultat attendu
`chercher([4, 8, 2], 8)`	`1`
`chercher([4, 8, 2], 2)`	`2`
`chercher([4], 4)`	`0`
`chercher([], 4)`	`None`

Repérez deux erreurs. Expliquez chacune avec un essai qui la révèle.

Corrigé : examiner toutes les positions utiles

```
def chercher(valeurs, cible):
    for i in range(len(valeurs)):
        if valeurs[i] == cible:
            return i
    return None
```

`range(len(valeurs))` inclut la dernière position valide. Le `return None` se trouve après la boucle : l'échec n'est certain qu'une fois toute la liste examinée.

Sur `[4, 8, 2]`, la recherche de 8 compare 4, puis 8, et renvoie 1. La recherche de 2 compare les trois valeurs.

Meilleur cas sur une liste non vide : $O(1)$. Pire cas : $O(n)$. Espace auxiliaire : $O(1)$. L'indice 0 est un résultat valide : testez l'absence avec `resultat is None`.

Complexité : que faut-il réellement compter ?

Les listes `catalogue` et `requetes` contiennent n et q identifiants de taille bornée. Chaque extrait part de `total = 0` et `reponses = []`.

```
# A
for i in range(n):
    for j in range(3):
        total += 1
# B
for cible in requetes:
    reponses.append(cible in catalogue)
# C
vus = set(catalogue)
for cible in requetes:
    reponses.append(cible in vus)
```

Pour B, la cible peut être absente. Pour C, utilisez le coût moyen du hachage. Comptez aussi la mémoire créée par `set(catalogue)`.

Corrigé : le travail d'une boucle dépend de son corps

Extrait	Opérations à compter	Temps
A	$3n$ incrémentations	$O(n)$
B	q recherches, jusqu'à n comparaisons chacune	$O(qn)$ au pire
C	Construction en $O(n)$, puis q tests moyens en $O(1)$	$O(n+q)$ en moyenne

C utilise jusqu'à $O(n)$ de mémoire auxiliaire pour l'ensemble. Les réponses occupent $O(q)$ dans B comme dans C.

À expliquer : remettre `set(catalogue)` dans la boucle ferait remonter le coût moyen à $O(qn)$. Une boucle imbriquée n'est donc pas toujours quadratique, et une ligne courte n'est pas toujours en $O(1)$.

Débogage objet : deux erreurs dans un club

Le club doit refuser une limite inférieure à 1 et posséder sa propre liste de membres. Pourquoi la création échoue-t-elle ? Après cette réparation, quel problème restera-t-il entre deux clubs ?

```
class Club:
    membres = []
    def __init__(self, limite):
        self.limite = limite
    @property
    def limite(self):
        return self._limite
    @limite.setter
    def limite(self, valeur):
        if valeur < 1:
            raise ValueError("limite invalide")
        self.limite = valeur
```

Corrigé : stockage interne et état propre à chaque objet

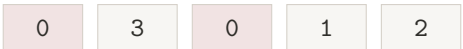
```
class Club:
    def __init__(self, limite):
        self.membres = []
        self.limite = limite
    @property
    def limite(self):
        return self._limite
    @limite.setter
    def limite(self, valeur):
        if valeur < 1:
            raise ValueError("limite invalide")
        self._limite = valeur
```

`_limite` évite que le setter s'appelle lui-même. `self.membres = []` crée une liste par instance. Le constructeur passe par le setter pour valider la limite initiale.

Testez deux clubs indépendants et vérifiez qu'un refus conserve l'ancienne limite.

Problème 1 : les zéros à la fin

But : créer une nouvelle liste avec tous les zéros à la fin. Les autres nombres gardent leur ordre.



Résultat attendu



La liste de départ reste inchangée. Nous écrivons `zeros_a_la_fin(valeurs)`.

Donnez aussi le résultat pour `[0, 0]`, `[4, 2]` et `[]`.

Intuition : garder les nombres, compter les zéros

On parcourt `[0, 3, 0, 1, 2]` de gauche à droite.

- Si on lit un nombre non nul, on l'ajoute au résultat.
- Si on lit un zéro, on augmente un compteur.

3	1	2

Après le parcours, le résultat contient `[3, 1, 2]` et le compteur vaut 2. Il reste à ajouter deux zéros.

À expliquer : pourquoi les nombres non nuls gardent-ils leur ordre ?

Les étapes à écrire

Écrivez le pseudocode. Utilisez une liste `resultat` et un compteur `nb_zeros`.

1. Avec quelles valeurs commencent-ils ?
2. Que faire quand le nombre lu vaut 0 ? Et sinon ?
3. Après le parcours, combien de zéros faut-il ajouter ?

Complétez cette trace avant de coder.

Nombre lu	resultat	`nb_zeros`
départ	`[]`	0
0	...	...
3	...	...
0	...	...
1	...	...
2	...	...

Corrigé du pseudocode

```
Créer une liste resultat vide
Mettre nb_zeros à 0
Pour chaque nombre de la liste :
    Si le nombre vaut 0 : augmenter nb_zeros
    Sinon : ajouter le nombre à resultat
Ajouter nb_zeros fois la valeur 0 à resultat
Renvoyer resultat
```

À coder : traduisez ces étapes dans `zeros_a_la_fin(valeurs)`. `append` ajoute une valeur à la fin d'une liste. `range(nb_zeros)` permet de répéter un ajout le nombre de fois voulu. Les nombres gardent leur ordre, car on les ajoute dans l'ordre de lecture.

Corrigé Python : deux boucles successives

```
def zeros_a_la_fin(valeurs):
    resultat = []
    nb_zeros = 0
    for nombre in valeurs:
        if nombre == 0:
            nb_zeros = nb_zeros + 1
        else:
            resultat.append(nombre)
    for i in range(nb_zeros):
        resultat.append(0)
    return resultat
```

`zeros_a_la_fin([0, 3, 0, 1, 2])` donne `[3, 1, 2, 0, 0]`.

Avant la suite, comptez les tours de chaque boucle. Sont-elles imbriquées ?

Trace, essais et coût

Nombre lu	resultat	`nb_zeros`
0	`[]`	1
3	`[3]`	1
0	`[3]`	2
1	`[3, 1]`	2
2	`[3, 1, 2]`	2

La seconde boucle ajoute les deux zéros. `[0, 0]`, `[4, 2]` et `[]` donnent respectivement `[0, 0]`, `[4, 2]` et `[]`.

La première boucle fait n tours, la seconde au plus n. On additionne ces travaux : temps $O(n)$. La nouvelle liste occupe $O(n)$ de mémoire.

Problème 2 : un nombre revient-il ?

But : répondre `True` si une liste contient au moins deux fois le même nombre.

Entrée	Résultat	Pourquoi
`[4, 2, 4]`	`True`	4 apparaît deux fois
`[4, 2, 7]`	`False`	Tous différents
`[]`	`False`	Aucune répétition

La fonction s'appellera `a_un_doublon(valeurs)`. Elle reçoit une liste d'entiers et ne la modifie pas.

À quel moment peut-on s'arrêter dans le premier exemple ?

Intuition : mémoriser les nombres déjà lus

Imaginez une feuille sur laquelle vous notez chaque nouveau nombre.

Nombre lu	Déjà sur la feuille ?	Ce qu'on fait
4	non	Noter 4
2	non	Noter 2
4	oui	Répondre True et arrêter

À écrire en pseudocode : que faut-il conserver ? Quel test effectuer avant d'ajouter un nombre ? Quand répondre `False` ?

Commencez avec une liste `vus`. Nous regarderons ensuite si une autre collection convient mieux.

Corrigé du pseudocode

```
Créer une liste vus vide
Pour chaque nombre :
    Si le nombre est déjà dans vus : renvoyer True
    Ajouter le nombre à vus
Après toute la boucle : renvoyer False
```

À coder : écrivez `a_un_doublon_liste(valeurs)`.

Tracez `vus` sur `[4, 2, 4]`, puis sur `[4, 2, 7]`. N'ajoutez pas le nombre avant de tester sa présence, sinon vous le retrouveriez toujours.

Corrigé : mémoriser dans une liste

```
def a_un_doublon_liste(valeurs):
    vus = []
    for nombre in valeurs:
        if nombre in vus:
            return True
        vus.append(nombre)
    return False
```

Avec `[4, 2, 4]`, `vus` devient `[4]`, puis `[4, 2]`. Le troisième nombre provoque le retour de `True`.

Question de coût : quand tous les nombres sont différents, `in` peut parcourir toute la liste `vus`. Pour quatre nombres, on examine jusqu'à $0 + 1 + 2 + 3 = 6$ valeurs déjà vues. Pour n nombres, le pire cas est $O(n^2)$.

Une petite modification pour chercher plus vite

Un **ensemble** conserve des valeurs distinctes. C'est une collection du chapitre 2.

Avec une liste	Avec un ensemble
`vus = []`	`vus = set()`
`vus.append(nombre)`	`vus.add(nombre)`
`nombre in vus`	`nombre in vus`

Le test de présence coûte $O(1)$ en moyenne dans l'ensemble. Nous n'avons pas besoin de l'ordre des nombres déjà lus.

À faire : adaptez la fonction avec ces deux changements. Nommez-la `a_un_doublon`. Vérifiez qu'elle donne les mêmes réponses.

Corrigé : un ensemble pour les nombres déjà vus

```
def a_un_doublon(valeurs):
    vus = set()
    for nombre in valeurs:
        if nombre in vus:
            return True
        vus.add(nombre)
    return False
```

`[4, 2, 4]` donne `True`. `[4, 2, 7]`, `[4]` et `[]` donnent `False`.

Avec les coûts moyens du hachage : n tours au plus, chaque test et ajout en $O(1)$. Le temps est $O(n)$ en moyenne. L'ensemble occupe jusqu'à $O(n)$ de mémoire.

Dans les deux versions, on teste d'abord, puis on mémorise. Nous allons réutiliser cette idée pour des salles.

Problème 3 : les mêmes lettres ?

Deux chaînes sont des **anagrammes** si elles contiennent les mêmes caractères, avec les mêmes nombres d'occurrences, dans un ordre éventuellement différent.

À construire : `anagrammes(s, t)` renvoie `True` ou `False`. Tous les caractères comptent : majuscules et minuscules sont distinctes, les espaces comptent aussi.

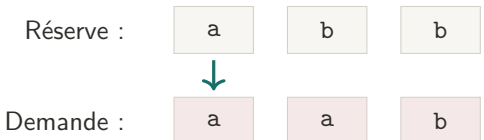


Entrées	Résultat
`"gare", "rage"`	`True`
`"abb", "aab"`	`False`
`"", ""`	`True`

Expliquez les résultats avant de choisir une collection.

Intuition : un exemplaire de chaque lettre ne suffit pas

À discuter : comparer `set(s)` et `set(t)` permet-il de résoudre le problème ? Dessinez une réserve de lettres pour `"abb"`, puis essayez d'en retirer les lettres de `"aab"`.



Il faut conserver des **quantités**. Un dictionnaire peut associer chaque caractère à son nombre d'exemplaires disponibles. Qu'apporte un test sur les longueurs avant de commencer ?

Anagrammes : votre pseudocode

Écrivez le pseudocode sans utiliser de tri.

1. Quel cas permet de répondre immédiatement ?
2. Comment constituer la réserve à partir de `s` ?
3. Que vérifier avant de retirer une lettre demandée par `t` ?
4. Si toutes les lettres ont été retirées et les longueurs sont égales, reste-t-il quelque chose à vérifier ?

Tracez ensuite les quantités sur `s = "abb"`, `t = "bab"`. Indiquez l'état de la réserve après chaque retrait.

Corrigé : constituer une réserve, puis la consommer

```
Si les longueurs diffèrent : renvoyer Faux
Compter chaque caractère de s dans un dictionnaire
Pour chaque caractère de t :
    Si sa quantité disponible est nulle : renvoyer Faux
    Diminuer sa quantité de 1
Renvoyer Vrai
```

Demande dans bab	Quantités restantes
Départ	a : 1, b : 2
b	a : 1, b : 1
a	a : 0, b : 1
b	a : 0, b : 0

À coder maintenant. Pensez à `get(caractere, 0)` lorsqu'une lettre n'existe pas dans la réserve.

Corrigé Python : anagrammes

```
def anagrammes(s, t):
    if len(s) != len(t):
        return False
    reserve = {}
    for lettre in s:
        reserve[lettre] = reserve.get(lettre, 0) + 1
    for lettre in t:
        if reserve.get(lettre, 0) == 0:
            return False
        reserve[lettre] -= 1
    return True
```

L'égalité des longueurs garantit que les retraits réussis consomment toute la réserve. Le programme ne change pas les chaînes reçues.

Ajoutez des tests avec une lettre répétée, des longueurs différentes, des espaces et une chaîne vide.

Anagrammes : tests et coût

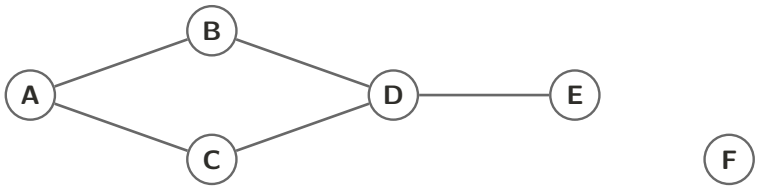
s	t	Résultat
`"abb"`	`"bab"`	`True`
`"abb"`	`"aab"`	`False`
`"a"`	`"aa"`	`False`
`"a b"`	`"ba "`	`True`
`"A"`	`"a"`	`False`
`""`	`""`	`True`

Avec n et m caractères : $O(n+m)$ en moyenne au plus, et $O(1)$ si les longueurs diffèrent. La réserve utilise $O(k)$ de mémoire pour k caractères distincts dans `s`.

Comparer des ensembles perd les fréquences : `"abb"` et `"aab"` donnent tous deux les lettres a et b.

Problème 4 : visiter toutes les salles accessibles

But : partir de A et visiter toutes les salles que l'on peut atteindre, sans visiter deux fois la même. Chaque trait représente une porte utilisable dans les deux sens.



La liste renvoyée donne l'ordre d'examen des salles. Ce n'est pas un trajet à pied : B et C peuvent se suivre dans la liste sans porte entre elles.

Sur ce dessin, quelles salles peut-on atteindre depuis A ? La salle F en fait-elle partie ?

Le plan fourni en Python

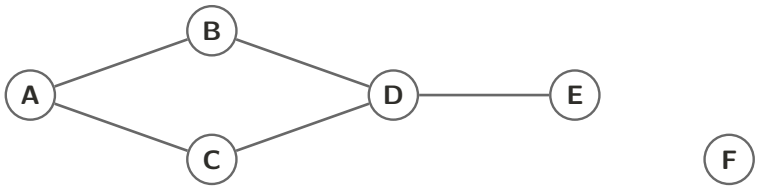
Un **graphe** représente des éléments et les liens entre eux. Ici : des salles reliées par des portes. Le dictionnaire donne les voisines de chaque salle.

```
plan = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C", "E"],
    "E": ["D"],
    "F": []
}
```

`plan["A"]` donne `["B", "C"]` : de A, on peut passer en B ou en C. F n'a aucune voisine.

Le plan est fourni. On lit les voisines dans l'ordre de leur liste. Le départ est toujours une salle du plan.

Le parcours à faire au tableau



On commence par A. On note B puis C dans une liste de salles à visiter.

1. On examine les portes de B. A est déjà connue. Quelle salle ajouter ?
2. C attendait avant D. On examine donc C ensuite. Faut-il ajouter D une deuxième fois ?
3. Quelle nouvelle salle découvre-t-on ensuite depuis D ?

On prend toujours la prochaine salle de la liste et on ajoute les nouvelles à la fin.

Corrigé : A, puis B, C, D et E

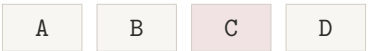
Salle examinée	Nouvelles salles à ajouter	Celles qui attendent encore
A	B, C	B, C
B	D	C, D
C	aucune	D
D	E	E
E	aucune	aucune

L'ordre obtenu est `["A", "B", "C", "D", "E"]`. F n'apparaît pas : aucune porte ne permet de la rejoindre.

Nous marquons une salle comme connue **dès que nous l'ajoutons**. Quand C retrouve D, D attend déjà son tour. Il ne faut donc pas l'ajouter à nouveau.

La liste d'attente et les salles déjà connues

Une liste `a_visiter` conserve les salles dans l'ordre. L'indice `position` indique laquelle examiner ensuite.



Après avoir examiné A puis B, `position` vaut 2 : la prochaine salle est C. On avance l'indice, sans supprimer les éléments précédents.

Un ensemble `vus` mémorise les salles déjà ajoutées. Il permet de tester rapidement `voisine in vus`.

Au départ : `a_visiter = [depart]`, `position = 0` et `vus = {depart}`. Les accolades contiennent ici un seul élément : c'est un ensemble.

Les étapes du parcours

La fonction `parcourir(plan, depart)` doit renvoyer la liste des salles visitées, dans l'ordre.

À écrire en pseudocode :

1. Tant qu'il reste une salle dans la liste, comment la lire ?
2. Pour chacune de ses voisines, comment reconnaître une salle encore inconnue ?
3. Si elle est nouvelle, dans quelles collections faut-il l'ajouter ?

L'idée est la même que pour les nombres déjà lus : tester la présence, puis mémoriser si c'est nouveau.

Corrigé du pseudocode

```
a_visiter = [départ]
position = 0
Mettre le départ dans l'ensemble vus
Tant que position est inférieure à la taille de a_visiter :
    Lire la salle à cette position
    Avancer position de 1
    Pour chaque voisine de cette salle :
        Si cette voisine n'est pas dans vus :
            L'ajouter dans vus
            L'ajouter à la fin de a_visiter
Renvoyer a_visiter
```

Le test évite de tourner en rond, par exemple en revenant de B vers A.

Le parcours : trois lignes à compléter

Remplacez les trois `...` : lire la prochaine salle, tester si une voisine est nouvelle, puis l'ajouter en fin de liste.

```
def parcourir(plan, depart):
    a_visiter = [depart]
    position = 0
    vus = {depart}
    while position < len(a_visiter):
        salle = ...
        position = position + 1
        for voisine in plan[salle]:
            if ...:
                vus.add(voisine)
                ...
    return a_visiter
```

Complétez le code, puis suivez A et B à la main avant d'exécuter la fonction.

Corrigé Python : visiter chaque salle une fois

```
def parcourir(plan, depart):
    a_visiter = [depart]
    position = 0
    vus = {depart}
    while position < len(a_visiter):
        salle = a_visiter[position]
        position = position + 1
        for voisine in plan[salle]:
            if voisine not in vus:
                vus.add(voisine)
                a_visiter.append(voisine)
    return a_visiter
```

`parcourir(plan, "A")` donne `["A", "B", "C", "D", "E"]`. `parcourir(plan, "F")` donne seulement `["F"]`.

Trace et coût du parcours

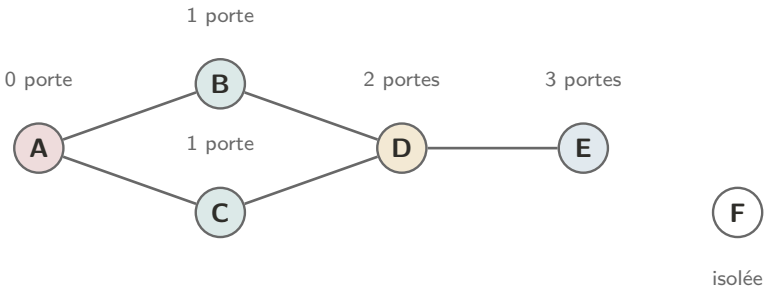
Après la salle	position	`a_visiter`
A	1	[A, B, C]
B	2	[A, B, C, D]
C	3	[A, B, C, D]
D	4	[A, B, C, D, E]
E	5	[A, B, C, D, E]

À la fin, l'indice vaut la longueur de la liste : la boucle s'arrête.

Chaque salle entre au plus une fois dans la liste. Chaque porte est examinée au plus deux fois. Pour V salles et E portes, on obtient $O(V + E)$ en moyenne, avec $O(V)$ de mémoire supplémentaire.

Les boucles sont imbriquées, mais la boucle intérieure ne parcourt que les portes de la salle actuelle.

En complément : compter les portes depuis A



Le départ est à 0 porte. Pour une nouvelle voisine : distance de la salle actuelle plus 1.

À adapter : `distances = {depart: 0}` remplace `vus`. Une clé présente signifie que la salle est déjà connue. Pour une nouvelle voisine, enregistrez `distances[salle] + 1`.

Donnez le pseudocode, puis adaptez la fonction pour renvoyer ce dictionnaire.

Corrigé : une distance pour chaque salle découverte

```
def distances_depuis(plan, depart):
    a_visiter = [depart]
    position = 0
    distances = {depart: 0}
    while position < len(a_visiter):
        salle = a_visiter[position]
        position = position + 1
        for voisine in plan[salle]:
            if voisine not in distances:
                distances[voisine] = distances[salle] + 1
                a_visiter.append(voisine)
    return distances
```

Depuis A : `{"A": 0, "B": 1, "C": 1, "D": 2, "E": 3}`. Il faut donc au minimum 3 portes pour rejoindre E. F reste absente, car elle est inaccessible depuis A.

Le parcours en largeur

Le parcours que vous venez de programmer s'appelle le **parcours en largeur**, ou **BFS**.

Il examine d'abord le départ, puis les salles à une porte, puis celles à deux portes, et ainsi de suite.

C'est aussi un algorithme de **plus court chemin** quand chaque passage a le même coût. En comptant les portes lors de la première découverte, on obtient les plus courtes distances depuis le départ.

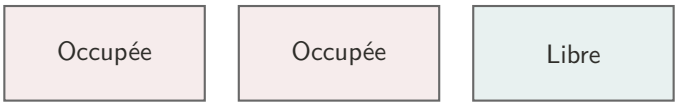
Pourquoi : si une salle était accessible en moins de portes, on l'aurait trouvée à un niveau précédent.

Avec des durées différentes selon les portes, le trajet qui traverse le moins de portes n'est pas forcément le plus rapide.

Source : MIT 6.006 : parcours en largeur

Problème 5 : les places libres d'un parking

But : un objet compte les voitures présentes dans un parking. Une entrée réussie occupe une place. Une sortie réussie en libère une.



Ici : capacité 3, voitures présentes 2, places libres 1.

Nous comptons seulement les voitures. Aucune plaque, heure d'arrivée ou facturation à gérer.

Ce que l'objet doit savoir faire

Utilisation	Comportement attendu
`p = Parking(3)`	Créer un parking vide de 3 places
`p.entrer()`	Ajouter une voiture s'il reste une place
`p.sortir()`	Retirer une voiture s'il y en a une
`p.places_libres`	Calculer capacité moins voitures présentes

`entrer` et `sortir` renvoient `True` si l'action réussit, sinon `False`. Un refus ne change rien.

La capacité reçue est un entier strictement positif. On suppose cette entrée correcte pour l'exercice.

Quelles deux valeurs faut-il conserver dans l'objet ?

Les données d'un parking

On conserve `self.capacite` et `self.occupes`. Au départ, `occupes` vaut 0.

Rappels du chapitre 4 :

- `__init__` initialise un objet lorsqu'on écrit `Parking(3)`.
- `self` désigne cet objet. `self.occupes` conserve sa valeur entre deux appels de méthodes.
- `@property` est un décorateur qui permet de lire le résultat d'une méthode comme un attribut, sans parenthèses.

À écrire : le pseudocode de l'initialisation, puis le constructeur. Ajoutez une propriété `places_libres` qui calcule `capacite - occupes`.

Corrigé : initialiser et lire les places libres

```
class Parking:
    def __init__(self, capacite):
        self.capacite = capacite
        self.occupes = 0

    @property
    def places_libres(self):
        return self.capacite - self.occupes
```

Pseudocode : conserver la capacité, mettre le nombre de voitures à 0. Pour les places libres, renvoyer leur différence.

Avec `p = Parking(3)`, `p.places_libres` donne 3. La propriété calcule le résultat à chaque lecture. On n'ajoute pas de setter, car on ne doit pas modifier directement ce résultat.

Entrer ou sortir : les conditions à écrire

Écrivez le pseudocode des deux méthodes, puis ajoutez-les à la classe.

entrer()	sortir()
Si le parking est plein : refuser Sinon : ajouter une voiture Renvoyer True si on a ajouté	Si le parking est vide : refuser Sinon : retirer une voiture Renvoyer True si on a retiré

Un refus renvoie `False`. Il faut vérifier la condition **avant** de modifier `self.occupes`.

Les voitures présentes doivent toujours rester entre 0 et la capacité. Testez notamment une sortie du parking vide et une entrée dans le parking plein.

Corrigé : deux méthodes courtes

```
def entrer(self):
    if self.occupes == self.capacite:
        return False
    self.occupes = self.occupes + 1
    return True

def sortir(self):
    if self.occupes == 0:
        return False
    self.occupes = self.occupes - 1
    return True
```

Placez ces méthodes **dans la classe Parking**, au même niveau que `__init__`. Réexécutez la définition complète, puis créez un nouvel objet pour les essais.

Parking : prévoir chaque résultat

On commence avec `p = Parking(2)`. Sans exécuter, complétez les deux colonnes.

Appel	Résultat	Places libres après
`p.sortir()`	...	...
`p.entrer()`	...	...
`p.entrer()`	...	...
`p.entrer()`	...	...
`p.sortir()`	...	...

Créez ensuite `autre = Parking(2)`. Combien de places libres possède ce deuxième parking ? Une entrée dans `p` doit-elle modifier `autre` ?

Corrigé : les refus conservent le compteur

Appel	Résultat	Places libres après
`p.sortir()`	`False`	2
`p.entrer()`	`True`	1
`p.entrer()`	`True`	0
`p.entrer()`	`False`	0
`p.sortir()`	`True`	1

`autre` possède 2 places libres : son constructeur a créé son propre compteur. Les deux objets évoluent indépendamment.

Chaque méthode effectue un nombre fixe de tests et de calculs : temps $O(1)$. Un parking conserve deux nombres : mémoire $O(1)$.

Autoévaluation

Choisissez un exercice et expliquez son fonctionnement avec un petit exemple.

- Je sais annoncer ce que reçoit et renvoie le programme.
- Je peux décrire les étapes avant de coder.
- Je peux suivre les variables à chaque tour de boucle.
- Je sais proposer un cas qui pourrait révéler une erreur.
- Je peux expliquer le coût sans me limiter au nombre de boucles.

Pour le parking, expliquez pourquoi le compteur ne doit jamais devenir négatif. Pour les salles, expliquez pourquoi on n'ajoute pas une salle déjà découverte.

En complément : une méthode qui change selon l'objet

Un ticket coûte 2 euros par heure. Un ticket gratuit doit toujours coûter 0.

```
class Ticket:
    def prix(self, heures):
        return 2 * heures
```

À écrire : le pseudocode du calcul pour un ticket gratuit, puis une classe `TicketGratuit(Ticket)` qui redéfinit `prix`. On suppose la durée positive ou nulle.

L'héritage permet à la classe fille de reprendre le comportement du parent. Redéfinir une méthode remplace ce comportement pour les objets de la classe fille.

Avant d'exécuter : que doivent donner `Ticket().prix(3)` et `TicketGratuit().prix(3)` ?

Corrigé : le même appel, deux comportements

```
class TicketGratuit(Ticket):
    def prix(self, heures):
        return 0

for ticket in [Ticket(), TicketGratuit()]:
    print(ticket.prix(3))
```

L'affichage est 6, puis 0. La boucle appelle la même méthode, mais Python choisit celle de l'objet concerné : c'est le **polymorphisme**.

Pseudocode du ticket gratuit : quelle que soit la durée reçue, renvoyer 0. La boucle n'a pas besoin de tester la classe de chaque ticket.